Q-Sim Classical QPU Studio

User Instruction Manual & Classical Quantum Simulation Guide

RELEASE V1.0 | STATEVECTOR ENGINE | NOISE & DECOHERENCE (T1/T2) | INTERACTIVE COMPOSER | TYPESCRIPT & REACT

Table of Contents

1. Overview & Architecture
- 1.1 System Architecture
- 1.2 State Representation & Bit Stride

2. Quick Start & Execution
- 2.1 Starting Backend & Frontend
- 2.2 Running Test Suites

3. Visual Interface Tour
- 3.1 Header & System Telemetry
- 3.2 Gate Palette & Categorization
- 3.3 Circuit Grid Canvas
- 3.4 Physical Hardware Noise Modal

4. Step-by-Step Tutorials
- 4.1 Tutorial 1: Bell State Entanglement
- 4.2 Tutorial 2: 3-Qubit GHZ State
- 4.3 Tutorial 3: Physical Noise Simulation
- 4.4 Tutorial 4: Toffoli Reversible Gate

5. REST API Developer Reference
- 5.1 POST /api/circuit/simulate
- 5.2 Programmatic cURL Usage

6. Troubleshooting & FAQ
- 6.1 Port Binding & Connectivity
- 6.2 Measurement Monte Carlo Variance

1. Overview & System Architecture

Q-Sim Classical QPU Studio is an end-to-end full-stack software application that mathematically simulates quantum circuit execution on classical hardware. The platform is decoupled into two independent layers:

- **Core Simulation Engine (`@quantum/engine`)**: A high-performance Node.js/TypeScript linear algebra simulator that handles unitary state-vector transformations, multi-qubit entangling gates, hardware noise channels (T_1, T_2, depolarizing, SPAM), and Monte Carlo projective measurement sampling.
- **Interactive Circuit Playground (`@quantum/web`)**: A modern React 18 / Vite front-end that provides an IBM Quantum Composer-style drag-and-drop circuit canvas, gate parameter inspectors, shot-distribution histograms (Recharts), and complex amplitude tables.

1.1 State-Vector Formulation & Cache Optimization

For an n-qubit register ($n \in [1, 16]$), the quantum state $|\psi\rangle$ is a complex column vector in $\mathbb{C}^{2^n}$: $$|\psi\rangle = \sum_{i=0}^{2^n-1} (a_i + i b_i) |i\rangle, \quad \sum_{i=0}^{2^n-1} (a_i^2 + b_i^2) = 1.0$$ The engine stores real (a_i) and imaginary (b_i) amplitudes in two contiguous `Float64Array` typed buffers. This avoids JavaScript object wrapper overhead and ensures optimal CPU L1/L2 cache locality during array traversals.

1.2 Bit-Stride In-Place Transformations

To eliminate exponential $\mathcal{O}(2^{2n})$ memory overhead from building dense Kronecker product matrices, single-qubit transformations on target qubit k are computed using a bit-stride algorithm:

```
// For basis index pairs differing only at bit position k (bit = 1 << k):
const i0 = i + j;
const i1 = i0 + bit;
// Complex matrix-vector multiply: [c0', c1']^T = U * [c0, c1]^T
real[i0] = (uR00 * r0 - uI00 * m0) + (uR01 * r1 - uI01 * m1);
imag[i0] = (uR00 * m0 + uI00 * r0) + (uR01 * m1 + uI01 * r1);
real[i1] = (uR10 * r0 - uI10 * m0) + (uR11 * r1 - uI11 * m1);
imag[i1] = (uR10 * m0 + uI10 * r0) + (uR11 * m1 + uI11 * r1);
```

2. Quick Start & Execution

2.1 Launching the Services

To run the application locally, open two PowerShell / terminal windows inside the project root:

Terminal 1: Start Backend Engine

```
cd "C:\Users\Niroop H\.gemini\antigravity\scratch\quantum-simulator"
npm run dev:engine
```

✓ Runs Express server on `http://localhost:4000`
✓ Health check available at `http://localhost:4000/health`

Terminal 2: Start Frontend Web Studio

```
cd "C:\Users\Niroop H\.gemini\antigravity\scratch\quantum-simulator"
npm run dev:web
```

✓ Vite development server starts on `http://localhost:3000`
✓ Open your browser to `http://localhost:3000` to access the visual composer.

2.2 Running Automated Test Suites

Execute the mathematical verification test suite at any time:

```
npm test
```

> **VERIFICATION RESULTS**
> All 10 core mathematical test suites pass with 100% accuracy, verifying Ground State initialization, Pauli-X, Hadamard, Bell State entanglement, 3-qubit GHZ state, Toffoli reversible logic, SWAP, continuous rotations, SPAM error confusion, and CDF sampling normalization.

3. Visual Interface Tour

3.1 Header Controls

Control	Default	Description & Operational Function
Backend Status	Online	Real-time telemetry showing virtual QPU status and active qubit register size.
Qubits Selector	3 Qubits	Dynamically scales register from 2 up to 8 wires on the visual canvas.
Shots Selector	1024	Configures Monte Carlo measurement repetitions ($512, 1024, 2048, 4096, 8192$).
Noise Toggle	Noise OFF	Toggles physical decoherence modeling. Button illuminates amber when active.
Reset (↻)	—	Clears all gates from wires and resets circuit to ground state $\|00\dots0\rangle$.
Run on QPU (▶)	—	Transmits circuit payload via JSON to the simulation engine and streams results.

3.2 Gate Palette Reference

Gate	Label	Category	Qubits	Mathematical Action / Function
H	H	Superposition	1	$H = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$. Maps $\|0\rangle \to \frac{\|0\rangle + \|1\rangle}{\sqrt{2}}$.
X	X	Clifford	1	Pauli-X (NOT). $X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$. Flips $\|0\rangle \leftrightarrow \|1\rangle$.
Y	Y	Clifford	1	Pauli-Y. $Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$. Bit-flip and phase rotation.
Z	Z	Clifford	1	Pauli-Z. $Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$. Phase flip on $\|1\rangle$.
S	S	Clifford	1	Phase Gate ($\pi/2$). $S = \text{diag}(1, i)$. $S^2 = Z$.
T	T	Phase	1	$\pi/4$ Gate. $T = \text{diag}(1, e^{i\pi/4})$. $T^2 = S$.
Rx	Rx	Parametric	1	$R_x(\theta) = \begin{bmatrix} \cos(\theta/2) & -i\sin(\theta/2) \\ -i\sin(\theta/2) & \cos(\theta/2) \end{bmatrix}$. Continuous rotation.
Ry	Ry	Parametric	1	$R_y(\theta) = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix}$. Real rotation matrix.
Rz	Rz	Parametric	1	$R_z(\lambda) = \text{diag}(e^{-i\lambda/2}, e^{i\lambda/2})$. Continuous Z phase rotation.
U	U	Parametric	1	Arbitrary unitary rotation $U(\theta, \phi, \lambda)$.
CX	CX	Entanglement	2	Controlled-NOT. Flips target qubit if control bit is 1.
CZ	CZ	Entanglement	2	Controlled-Z. Applies phase flip if both qubits are in state $\|11\rangle$.
SWAP	SWP	Entanglement	2	Exchanges states: $\|01\rangle \leftrightarrow \|10\rangle$.
CCX	CCX	Multi-Qubit	3	Toffoli Gate. Flips target qubit if both control qubits are 1.
M	M	Operations	1	Measurement operator on computational basis.

3.3 Circuit Grid Canvas Interaction

- **Adding a Gate**: Select a gate from the palette, then click any empty slot in the wire grid.
- **Multi-Qubit Linking**: Clicking a target wire for `CX`, `CZ`, `SWAP`, or `CCX` automatically generates the SVG link lines and control nodes on adjacent wires.
- **Deleting a Gate**: Hover the mouse cursor over any placed gate. Click the red trashcan icon that appears in the top-right corner.

3.4 Physical Hardware Noise Modal

Simulates physical quantum processor imperfections rather than an idealized mathematical state:

- **T_1 Thermal Relaxation**: Spontaneous decay rate $|1\rangle \to |0\rangle$ with probability $\gamma = 1 - e^{-t_g / T_1}$.
- **T_2 Pure Dephasing**: Loss of quantum phase coherence $\lambda = 1 - e^{-t_g / T_\phi}$.

- **Depolarizing Noise**: Injects random Pauli error $\{X, Y, Z\}$ with rate p.
- **Readout Confusion (SPAM)**: Matrix $P(\text{measured} \mid \text{actual})$ modeling detector false positives/negatives.

4. Step-by-Step Circuit Tutorials

1 Tutorial 1: Creating a Maximally Entangled Bell State ($|\Phi^+\rangle$)

The Bell State produces non-local entanglement between two qubits.

1. Set **Qubits** to `2` in the top header.
2. Select the **H** gate from the palette; click slot 0 on wire `q[0]`.
3. Select the **CX** gate from the palette; click slot 1 on wire `q[1]`.
4. Set **Shots** to `4096` and click **Run on QPU**.

> **Theoretical Expectation**: $|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$.
> Histogram displays $\approx 50\%$ for $|00\rangle$ and $\approx 50\%$ for $|11\rangle$. States $|01\rangle$ and $|10\rangle$ have 0% probability.

2 Tutorial 2: Composing a 3-Qubit GHZ State ($|\text{GHZ}\rangle$)

Entangles three distinct quantum wires into a collective superposition.

1. Set **Qubits** to `3`.
2. Place an **H** gate on wire `q[0]` at column 0.
3. Place a **CX** gate on wire `q[1]` at column 1 (controlled by q_0).
4. Place a **CX** gate on wire `q[2]` at column 2 (controlled by q_1).
5. Click **Run on QPU**.

> **Theoretical Expectation**: $|\text{GHZ}\rangle = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$.
> Outcomes will be evenly divided between $|000\rangle$ ($\approx 50\%$) and $|111\rangle$ ($\approx 50\%$).

3 Tutorial 3: Testing Decoherence on Superconducting Transmons

Demonstrates the difference between mathematical simulation and real hardware physics.

1. Load or compose the Bell state circuit ($[H(0), CX(0 \to 1)]$).
2. Click **Noise OFF** in the header bar.
3. Check **Enable QPU Noise Simulation**. Leave default values ($T_1 = 50\,\mu\text{s}$, $T_2 = 70\,\mu\text{s}$, Readout Error $\sim 2\%$).
4. Click **Apply & Close**, then click **Run on QPU**.

> **Observed Hardware Jitter**: Primary peaks remain at $|00\rangle$ and $|11\rangle$ ($\sim 47\text{–}48\%$), but small error peaks appear at $|01\rangle$ and $|10\rangle$ ($\sim 1\text{–}2\%$) due to physical SPAM and relaxation channels. Estimated fidelity drops to $\sim 98\%$.

4 Tutorial 4: Universal Reversible Logic with the Toffoli Gate (CCX)

Tests controlled-controlled NOT gate execution.

1. Set **Qubits** to `3`.
2. Place **X** on `q[0]` and **X** on `q[1]` (initializes inputs to $|110\rangle$).
3. Place **CCX** on `q[2]` at column 2.
4. Click **Run on QPU**.

> **Theoretical Expectation**: Target q_2 flips from $0 \to 1$ because both controls are active. Result: 100% measured as $|111\rangle$.

5. REST API Developer Reference

5.1 POST `/api/circuit/simulate`

The backend accepts standard JSON payloads for automated simulation workflows.

Request JSON Payload Schema

```
{
  "qubits": 2,
  "shots": 2048,
  "steps": [
    { "id": "step-1", "gate": "H", "targets": [0] },
    { "id": "step-2", "gate": "CX", "targets": [1], "controls": [0] }
  ],
  "noiseModel": {
    "enabled": false,
    "t1": 50.0,
    "t2": 70.0,
    "gateTime": 0.02,
    "readoutError": [0.015, 0.025],
    "depolarizingRate": 0.001
  }
}
```

Programmatic cURL Execution

```
curl -X POST http://localhost:4000/api/circuit/simulate \
  -H "Content-Type: application/json" \
  -d '{"qubits":2,"shots":1024,"steps":[{"id":"1","gate":"H","targets":[0]},{"id":"2","gate":"CX","tar
```

Response JSON Payload

```
{
  "counts": { "00": 519, "11": 505 },
  "probabilities": { "00": 0.506836, "11": 0.493164 },
  "stateVector": [
    { "state": "|00>", "real": 0.707107, "imag": 0, "magnitude": 0.707107, "phase": 0 },
    { "state": "|11>", "real": 0.707107, "imag": 0, "magnitude": 0.707107, "phase": 0 }
  ],
  "executionTimeMs": 1.45,
  "backendInfo": {
    "qpuSimulated": "Ideal_Statevector_Engine",
    "qubitCount": 2,
    "shots": 1024,
    "fidelityEstimated": 1.0
  }
}
```

6. Troubleshooting & FAQ

Issue / Symptom	Probable Cause	Resolution
"Failed to fetch" alert	Backend simulation engine is offline.	Run `npm run dev:engine` and confirm `http://localhost:4000/health` responds with status `"healthy"`.
Port 4000 or 3000 busy	Another process occupies the default port.	Terminate orphaned processes using `Stop-Process -Id <PID>` or specify `PORT=4001`.
Shot counts vary slightly	Monte Carlo projective sampling variance.	Quantum measurements follow statistical multinomial distributions. Set shots to 8192 for lower variance ($\sigma \propto 1/\sqrt{N}$).
Maximum register limit	Browser DOM canvas rendering density.	Canvas UI supports 2–8 qubits. Programmatic API supports up to 16 qubits ($2^{16} = 65,536$ state components).